

Apuntes de Python (PDF pasado a L^AT_EX)

Índice general

1. Tipos de variable	4
2. Operadores	4
2.1. Operadores aritméticos	4
2.2. Operadores lógicos	4
3. Operadores condicionales	4
3.1. if / elif / else	4
3.2. match / case	5
4. Operaciones con String	5
5. Tipos de objetos: listas y tuplas	5
5.1. Operaciones con listas	5
5.2. Recorrer dos listas a la vez	6
5.3. De lista a string (inverso de split)	6
5.4. Recorridos con índices	6
5.5. f-strings	6
5.6. break y continue	6
5.7. Tupla de un solo elemento	6
5.8. Zip / Unzip	6
6. Lista con for comprimida (comprensiones)	7
7. Diccionarios	7
8. Conjuntos o sets	8
9. Input y Print	8
10. Funciones	8
10.1. range()	8
10.2. *args y **kwargs	9
10.3. Ejemplos de funciones	9
11. Funciones anónimas (lambda) y utilidades	9
11.1. Lambda	9
11.2. map, filter, sorted	9
11.3. Funciones map, filter y sorted	9
12. Alcance (Scope) y Namespaces	10

13. Clases y objetos (no va a examen)	10
13.1. Conceptos	10
13.2. Ejemplo de clase	10
13.3. Herencia y polimorfismo	10
13.4. Encapsulamiento y abstracción	11
14. Archivos	11
14.1. Abrir archivos	11
14.2. Leer y escribir	11
14.3. CSV	12
14.4. JSON	12
14.5. Imágenes (PIL)	12
15. Compresiones de datos	12
16. Uso de secuencias	12
17. Excepciones y errores	13
17.1. raise y try/except/else/finally	14
17.2. Errores personalizados	14
18. Generadores	14
19. NumPy	14
19.1. Introducción	14
19.2. Inicialización típica	15
19.3. Parámetros importantes	15
19.4. Funciones	15
19.5. Matrices	16
19.6. Más operaciones	16
19.7. Resolver sistemas	17
19.8. where	17
19.9. Slicing	17
19.10 Slicing condicional	17
19.11 Random	18
19.12 Archivos	18
20. Pandas DataFrame	18
20.1. Definición	18
20.2. Series	18
20.3. Slicing en Series	19
20.4. Iteración	19
20.5. DataFrame	19
20.6. Archivos en Pandas	21
20.7. Selección y filtrado, groupby	21
20.8. Correlaciones	22
21. Visualization (Matplotlib)	22
21.1. Representación de más de 1 gráfico en una subfigura	25
21.2. Anotaciones y tipos de gráficos	26
21.3. Tipos de gráficos	27
21.4. Pandas plotting	30

22.Seaborn	31
23.Procesamiento de datos	42
23.1. Combinar datasets	42
23.2. Agrupaciones	43
23.3. Manipulación de strings	43
23.4. Expresiones regulares	43
23.5. Operaciones con colecciones: functools	43
24.Transformación de variables (Pandas/DataFrame)	43
24.1. Duplicados	43
25.Examen: Generadores	43
26.For en una línea	44

1. Tipos de variable

- **Int**
- **Float**
- **Boolean**: true y false
- **Str**: cadena de texto
- **Complex**: número complejo ($2+3j$)

Si queremos convertir **str** a **int**:

```
B = int(A) # siendo A un str
```

2. Operadores

2.1. Operadores aritméticos

- + suma
- - resta
- * multiplicación
- / división
- // parte entera de la división
- % módulo
- ** potencia (ej.: $3**2 = 9$)

2.2. Operadores lógicos

- **and**: si ambas condiciones se cumplen
- **or**: si se cumple una de ellas o ambas
- **not**: con booleanos, cambia el valor booleano

Ejemplo con **not**:

```
llueve = False
if not llueve:
    print("no esta lloviendo")
else:
    print("esta lloviendo")
```

3. Operadores condicionales

3.1. if / elif / else

```
if condicion:
    ...
elif otra_condicion:
    ...
else:
    ...
```

3.2. match / case

Para elegir opciones:

```
menu = 1
match menu:
    case 1:
        print("Ha elegido el menú numero 1")
    case 2:
        print("Ha elegido el menú numero 2")
    case _:
        print("menú no valido")
```

4. Operaciones con String

- `len()` (length): longitud de la cadena
- Se pueden sumar cadenas para concatenarlas (por ejemplo, añadiendo un espacio con " ")
- `upper()` / `lower()` para mayúsculas / minúsculas
- `capitalize()` primera letra en mayúscula

5. Tipos de objetos: listas y tuplas

Listas [] y tuplas ().

Ejemplo:

```
lista = [1, 43, 54, 2, 4]
tupla = (1, 2, 3)
```

- `split()` separa una cadena de texto en listas
- `strip()` elimina espacios u otros caracteres pasados como argumento
- Almacenan datos en un orden específico (importa el orden)
- La lista es mutable; la tupla es inmutable

5.1. Operaciones con listas

```
L1.append(x)           # añade al final
L1.insert(pos, dato)   # añade en cierta posición
L1.remove(dato)        # elimina el dato (no por posición)
del L1[1]              # elimina por posición
L1.pop()               # elimina el último
dato = L1.pop()        # guarda el último y lo elimina
```

Se puede pasar de `string` a lista: `list(string)`.

Última posición: índice -1.

5.2. Recorrer dos listas a la vez

```
lista1 = [1, 2, 3]
lista2 = ['a', 'b', 'c']
for x, y in zip(lista1, lista2):
    print(x, y)
```

Si hacemos `lista2 = lista1`, ambas apuntan al mismo espacio de memoria. Para evitarlo:

```
lista3 = lista1.copy()
```

5.3. De lista a string (inverso de split)

```
" ".join(lista3)
```

5.4. Recorridos con índices

```
for i in range(len(lista)):
    print(lista[i])
```

Ejemplo de enumerate:

```
frutas = ["manzana", "pera", "uva"]

for indice, valor in enumerate(frutas):
    print(indice, valor)
```

5.5. f-strings

```
print(f"..... {variable} .....")
```

5.6. break y continue

- `break` termina el bucle
- `continue` pasa a la siguiente iteración

5.7. Tupla de un solo elemento

```
t = (5,)
```

5.8. Zip / Unzip

- `zip` combina objetos múltiples como listas o tuplas, emparejando sus elementos.
- `Unzip` hace lo contrario: desempaqueta una estructura combinada en varias secuencias.

Ejemplo de zip:

```
nombres = ["Ana", "Luis", "María"]
edades = [20, 25, 30]

combinado = list(zip(nombres, edades))
print(combinado)
# [('Ana', 20), ('Luis', 25), ('María', 30)]
```

Ejemplo de unzip:

```
combinado = [('Ana', 20), ('Luis', 25), ('María', 30)]

nombres, edades = zip(*combinado)
print(nombres)  # ('Ana', 'Luis', 'María')
print(edades)   # (20, 25, 30)
```

6. Lista con for comprimida (comprensiones)

```
[x for x in range(10) if x % 2 == 0]  # pares del 0 al 9

[0 if x % 2 == 0 else x for x in range(10)]
# lista del 0 al 9, pares convertidos a 0
```

7. Diccionarios

Ejemplo:

```
diccionario = {"clave1": 3, "clave2": 15, "clave3": 7}
valor = diccionario["clave1"]  # 3
```

Modificar valores:

```
diccionario["clave1"] = 10
```

Añadir:

```
diccionario["clave4"] = 11
```

get con valor por defecto:

```
diccionario.get("clave1", "valor no encontrado")
```

pop:

```
edad = diccionario.pop("clave3")
```

items, keys, values:

```
# Recorre el diccionario devolviendo pares (clave, valor)
for clave, valor in diccionario.items():
    # Imprime cada clave junto con su valor asociado
    print(f"Clave: {clave}, Valor: {valor}")

# Devuelve una vista con todas las claves del diccionario
diccionario.keys()

# Devuelve una vista con todos los valores del diccionario
diccionario.values()

# Devuelve una vista con pares (clave, valor) del diccionario
diccionario.items()
```

Eliminar clave:

```
del diccionario["edad"]
```

Crear diccionario desde dos listas:

```
dict(zip(lista1, lista2))
```

Crear lista a partir de un diccionario:

```
listadedatos = [[nombre] + datos for nombre, datos in dhiper.items()]
```

8. Conjuntos o sets

Son colecciones sin posiciones; útiles para eliminar duplicados y hacer operaciones.

Set vacío:

```
set_vacio = set() # no usar {} porque eso es diccionario vacío
```

Operaciones:

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}

set1 | set2 # union
set1 & set2 # interseccion
set1 - set2 # diferencia
```

Métodos:

```
# Añade el elemento 20 al conjunto (si ya existe, no ocurre nada)
conjunto.add(20)

# Añade varios elementos al conjunto a partir de otro iterable
# (los duplicados se ignoran automáticamente)
conjunto.update([7, 8, 9])

# Elimina el elemento 20 del conjunto
# Lanza un error si el elemento no existe
conjunto.remove(20)
```

Diferencia lista vs set:

- En listas, los datos no son únicos y el orden importa.
- En sets, los datos son únicos y el orden no importa.

9. Input y Print

input pide un valor al usuario y lo asigna a una variable.

```
nombre = input("Ingrese su nombre: ")
print("Nombre", nombre, sep="--")
print(f"Nombre {nombre}") # más fácil
```

10. Funciones

10.1. range()

Secuencias:

- range(stop)
- range(start, stop)
- range(start, stop, step)

10.2. *args y **kwargs

- ***x**: recibe número indeterminado de argumentos (tupla)
- ****x**: recibe argumentos clave-valor (diccionario)

10.3. Ejemplos de funciones

```
def saludo(nombre, ciudad="Madrid"):
    return nombre, ciudad
```

```
def frutas(*frutas):
    return frutas # tupla

def frutas_dict(**frutas):
    return frutas # diccionario
```

Devolver múltiples resultados:

```
def operaciones(a, b):
    suma = a + b
    resta = a - b
    multiplicacion = a * b
    return suma, resta, multiplicacion

suma, resta, multiplicacion = operaciones(2, 5)
```

Devolver diccionario:

```
def operaciones_dict(a, b):
    suma = a + b
    resta = a - b
    multiplicacion = a * b
    return {"suma": suma, "resta": resta, "multiplicacion": multiplicacion}
```

11. Funciones anónimas (lambda) y utilidades

11.1. Lambda

Sintaxis: lambda argumentos: expresión

```
lambda x: x**2
```

11.2. map, filter, sorted

11.3. Funciones map, filter y sorted

map aplica una función a cada elemento de un iterable y devuelve un objeto iterable con los resultados.

```
numeros = [1, 2, 3, 4]

# Aplica la función x**2 a cada elemento de la lista
# Resultado: [1, 4, 9, 16]
cuadrados = list(map(lambda x: x**2, numeros))
```

filter selecciona únicamente los elementos que cumplen una condición y devuelve un iterable con dichos elementos.

```
# Filtra los elementos pares de la lista
# Resultado: [2, 4]
pares = list(filter(lambda x: x % 2 == 0, numeros))
```

sorted ordena los elementos de un iterable y permite especificar un criterio de ordenación mediante una función.

```
palabras = ["manzana", "uva", "banana", "kiwi"]

# Ordena las palabras según su longitud
# key indica el criterio de ordenación
# Resultado: ['uva', 'kiwi', 'manzana', 'banana']
ordenadas = sorted(palabras, key=len)
```

12. Alcance (Scope) y Namespaces

Si hay una variable global y variables locales anidadas, se puede usar **nonlocal x** para referirse a la variable del ámbito inmediatamente superior (se recomienda evitarlo y usar nombres distintos o parámetros).

13. Clases y objetos (no va a examen)

13.1. Conceptos

Clase = plantilla (ej. Persona). Objeto = instancia (ej. Juan).

13.2. Ejemplo de clase

```
class Empleado:
    empresa = "Google" # compartido

    def __init__(self, nombre_empleado):
        self.nombre = nombre_empleado
```

Acceso:

- Si no está definido con **self**, se comparte por instancias: `empleado1.empresa`
- *init* se ejecuta al crear el objeto

13.3. Herencia y polimorfismo

```
class Clase_Principal:
    pass

class Subclase(Clase_Principal):
    pass
```

Para usar métodos de la clase padre desde la hija: `super().metodo()`.

13.4. Encapsulamiento y abstracción

Convención: variables con `__` se consideran protegidas.

Clases abstractas:

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def hacer_sonido(self):
        pass
```

Propiedades:

```
@property
def name(self):
    return self.__name

@name.setter
def name(self, value):
    self.__name = value
```

14. Archivos

14.1. Abrir archivos

```
import os # añade la función path
file_path = ["res", "prueba.txt"]
ruta = os.path.join(*file_path)

archivo = open(ruta, "r")
archivo.close()
```

Con `with` se cierra automáticamente:

```
with open("prueba.txt", "r") as archivo:
    contenido = archivo.read()
    print(contenido)
```

14.2. Leer y escribir

- `archivo.read()` permite leer el contenido completo del archivo.
- `archivo.write()` escribe en el archivo, sobrescribiendo el contenido desde la posición actual del cursor.

Modos de acceso a archivos en Python:

- `r`: Solo lectura. El archivo debe existir.
- `w`: Solo escritura. Borra el contenido si el archivo existe y lo crea si no existe.
- `x`: Solo escritura. Falla si el archivo ya existe.
- `a`: Escritura al final del archivo. Si existe, añade contenido; si no, lo crea.
- `r+`: Lectura y escritura. El archivo debe existir.
- `b`: Modo binario. Se añade a otros modos (por ejemplo `rb`, `wb`).
- `t`: Modo texto. Es el modo por defecto.

14.3. CSV

```
import csv

with open("datos.csv", mode="w", newline="") as archivo:
    escritor = csv.writer(archivo)
    escritor.writerow(["nombre", "edad", "ciudad"])
    escritor.writerow(["Alice", "30", "Nueva York"])
    escritor.writerow(["Juan", "34", "Quito"])

with open("datos.csv", mode="r") as archivo:
    lector = csv.reader(archivo)
    for fila in lector:
        print(fila)
```

14.4. JSON

```
import json

with open("datos.json", "w") as archivo:
    json.dump({"nombre": "Alice", "edad": 30}, archivo)

with open("datos.json", "r") as archivo:
    datos = json.load(archivo)
    print(datos)
```

14.5. Imágenes (PIL)

```
from PIL import Image

imagen = Image.open("flor.jpg")
imagen.show()
imagen_gris = imagen.convert("L")
imagen_gris.save("flor_gris.jpg")
```

15. Compresiones de datos

```
# [nueva_expresion for elemento in iterable if condicion]
cuadrados = [x**2 for x in range(10) if x % 2 == 0]
print(cuadrados)

# {nueva_expresion for elemento in iterable if condicion}
cuadrados_set = {x**2 for x in range(10)}
print(cuadrados_set)

# {clave: valor for elemento in iterable if condicion}
cuadrados_dict = {x: x**2 for x in range(1, 5)}
print(cuadrados_dict)
```

16. Uso de secuencias

Desempaquetado:

```
primero, *resto = [1, 2, 3, 4, 5] # Hace que en primero se guarde un 1 y en
    resto se guarden el resto
```

Crear una matriz con comprensiones:

```
matriz = [[j**2 for j in range(4)] for i in range(4)]
print(matriz)
```

Combinar listas:

```
import itertools
combinado = list(itertools.chain([1, 2, 3], [4, 5, 6]))
```

17. Excepciones y errores

Lista de excepciones comunes (resumen del PDF):

- **BaseException:** Es la clase base de todas las excepciones integradas en Python. No está pensada para ser capturada directamente.
- **SystemExit:** Se lanza cuando se llama a la función `sys.exit()` y provoca la finalización del programa.
- **KeyboardInterrupt:** Se lanza cuando el usuario interrumpe manualmente la ejecución del programa, normalmente pulsando `Ctrl + C`.
- **Exception:** Clase base para la mayoría de las excepciones que no están relacionadas con el sistema. La mayoría de los errores comunes heredan de esta clase.
- **StopIteration:** Se lanza automáticamente por los iteradores para indicar que no quedan más elementos que recorrer.
- **ArithmeticError:** Clase base para errores en operaciones aritméticas. No se usa directamente, pero tiene subclases como:
 - **ZeroDivisionError:** Se lanza cuando se intenta dividir entre cero.
 - **OverflowError:** Se lanza cuando el resultado de una operación es demasiado grande para ser representado.
 - **FloatingPointError:** Se lanza cuando ocurre un error en una operación de punto flotante.
- **LookupError:** Clase base para errores relacionados con búsquedas. No se usa directamente, pero tiene subclases como:
 - **IndexError:** Se lanza cuando un índice no es válido para una secuencia.
 - **KeyError:** Se lanza cuando una clave no existe en un diccionario.
- **NameError:** Se lanza cuando se intenta acceder a una variable que no está definida ni en el ámbito local ni en el global.
- **TypeError:** Se lanza cuando se aplica una operación o función a un objeto de un tipo inapropiado.
- **ValueError:** Se lanza cuando una función recibe un argumento del tipo correcto, pero con un valor no adecuado.

- `FileNotFoundError`: Se lanza cuando una operación sobre archivos falla porque el archivo no existe.
- `PermissionError`: Se lanza cuando una operación sobre archivos falla debido a falta de permisos.

Se usa `raise` `CualquierError` para hacer errores personalizados

17.1. `raise` y `try/except/else/finally`

```
try:
    # codigo
    pass
except ZeroDivisionError:
    # si ocurre el error
    pass
else:
    # si no ocurre el error
    pass
finally:
    # siempre se ejecuta al final
    pass
```

17.2. Errores personalizados

```
class MiError(Exception):
    """Excepción personalizada."""
    def __init__(self, mensaje):
        self.mensaje = mensaje
        super().__init__(self.mensaje)

def verificar_valor(valor):
    if valor < 0:
        raise MiError("El valor no puede ser negativo.")
    else:
        print("Valor válido:", valor)

try:
    verificar_valor(-10)
except MiError as e:
    print("Error:", e)
```

18. Generadores

Mejor que listas si queremos valores a demanda (eficiencia de memoria).

19. NumPy

19.1. Introducción

- Import típico: `import numpy as np`
- Array multidimensional (n-dimensiones) de tipo fijo, implementado en C
- Base de Pandas, SciPy, Scikit-learn, etc.

19.2. Inicialización típica

- `np.zeros(shape, dtype, order)`: Crea un array de NumPy con la forma indicada, inicializado con ceros.
- `np.ones(shape, dtype, order)`: Crea un array de NumPy con la forma indicada, inicializado con unos.
- `np.empty(...)`: Crea un array sin inicializar. Su contenido es aleatorio (depende de la memoria), pero es más rápido que `zeros` u `ones`.
- `np.full((5,2), 2)`: Crea un array de tamaño 5×2 donde todos los elementos valen 2.
- `np.eye(n)`: Crea una matriz identidad de tamaño $n \times n$, con unos en la diagonal principal y ceros en el resto.
- `np.array(lista)`: Convierte una lista (o estructura similar) de Python en un array de NumPy.
- `np.arange(100)`: Genera un array con valores desde 0 hasta 99, con incremento 1.
- `np.arange(2, 10, 2.1)`: Genera un array de valores desde 2 hasta 10 (sin incluir 10), con incrementos de 2.1.

Ejemplos:

```
import numpy as np

lista_100 = np.arange(100)
lista_100_2 = lista_100 * 2

matriz = np.array([[1, 2, 3], [4, 5, 6]])
```

19.3. Parámetros importantes

- **shape**: Devuelve la forma del array, es decir, el número de elementos en cada dimensión (por ejemplo, filas y columnas).
- **size**: Devuelve el número total de elementos del array.
- **dtype**: Indica el tipo de dato de los elementos del array (entero, flotante, etc.).
- **order**: Indica cómo se almacenan los datos en memoria (por filas, orden C, o por columnas, orden F).

19.4. Funciones

Element-wise (operan elemento a elemento):

- `np.add`: Suma dos arrays elemento a elemento.
- `np.subtract`: Resta dos arrays elemento a elemento.
- `np.multiply`: Multiplica dos arrays elemento a elemento.
- `np.divide`: Divide dos arrays elemento a elemento.
- `np.mod`: Calcula el resto de la división elemento a elemento.

- `np.divmod`: Devuelve el cociente y el resto de la división elemento a elemento.

Álgebra y estadística:

- `np.dot`: Calcula el producto escalar entre vectores o el producto matricial entre matrices.
- `np.sum`: Calcula la suma de los elementos del array.
- `np.mean`: Calcula la media aritmética de los elementos del array.
- `np.var`: Calcula la varianza de los elementos del array.
- `np.std`: Calcula la desviación estándar de los elementos del array.
- `np.median`: Calcula la mediana de los elementos del array.
- `np.trace`: Calcula la suma de los elementos de la diagonal principal de una matriz.

Utilidades:

- `np.clip`: Limita los valores del array a un rango mínimo y máximo.
- `np.round`: Redondea los valores del array al número de decimales indicado.
- `np.floor`: Redondea los valores del array hacia abajo.
- `np.ceil`: Redondea los valores del array hacia arriba.
- `np.trunc`: Elimina la parte decimal de los valores del array.

19.5. Matrices

```
a = np.eye(3)
# Multiplicacion elemento a elemento: a*b
# Multiplicacion matricial:
# np.dot(a, b)
```

19.6. Más operaciones

```
# Devuelve el índice del valor máximo del array
array.argmax()

# Devuelve el valor máximo del array
array.max()

# Intercambia los ejes 0 y 1 del array (por ejemplo, filas por columnas)
array.swapaxes(0, 1)

# Cambia la forma del array sin modificar sus datos
# Los puntos suspensivos indican la nueva forma deseada
array.reshape(...)

# Invierte el orden de los elementos del array a lo largo del eje indicado
# axis=0 invierte el orden de las filas
np.flip(array, axis=0)

# Une los arrays v1 y v2 horizontalmente (por columnas)
np.hstack((v1, v2))
```



```
# Une los arrays v1 y v2 verticalmente (por filas)
np.vstack((v1, v2))

# Divide el array en 2 subarrays a lo largo del eje 0
np.array_split(array, 2, axis=0)

# Devuelve True si al menos un elemento del array es True
array.any()

# Devuelve True solo si todos los elementos del array son True
array.all()
```

19.7. Resolver sistemas

```
import numpy.linalg as la
# la.solve(A, b)
```

19.8. where

```
np.where(precios < 1, 1.0, precios)    # reemplazo condicional
np.where(precios < 1)                  # indices donde se cumple
```

19.9. Slicing

En NumPy, el slicing devuelve una **referencia** al array original (no una copia). Para crear una copia independiente se debe usar `.copy()`.

```
import numpy as np

array = np.array([1, 2, 3, 4, 5])

# Slicing: devuelve una referencia
subarray = array[1:4]

# Modificar el subarray modifica el array original
subarray[0] = 99

print(array)
# Resultado: [ 1 99  3  4  5 ]
```

19.10. Slicing condicional

El slicing condicional devuelve una **copia** del array, no una referencia.

```
import numpy as np

array = np.array([1, 2, 3, 4, 5])

# Slicing condicional: devuelve una copia
subarray = array[array > 3]

# Modificar el subarray NO modifica el array original
subarray[0] = 99
```

```
print(array)
# Resultado: [1 2 3 4 5]
```

19.11. Random

```
import random

# Devuelve un número decimal aleatorio entre 0 y 1
random.random()

# Devuelve un número entero aleatorio entre 0 y 5 (ambos incluidos)
random.randint(0, 5)

# Devuelve un número decimal aleatorio entre 0 y 5
random.uniform(0, 5)

import numpy as np

# Devuelve un número decimal aleatorio entre 0 y 1
np.random.rand()

# Devuelve un array de tamaño 2x4 con enteros aleatorios entre 0 y 4
np.random.randint(5, size=(2, 4))
```

19.12. Archivos

- `np.load(ruta)`
- Mejor guardar en `.npy` (binario, ocupa menos)

20. Pandas DataFrame

20.1. Definición

- Estructura tabular 2D construida sobre NumPy.
- Columnas con nombres, índices para filas, tipos por columna.

```
import pandas as pd

df = pd.DataFrame({
    "Nombre": ["Ana", "Luis", "Maria"],
    "Edad": [28, 34, 22]
})
```

20.2. Series

Datos unidimensionales: elemento + índice. Se puede especificar índice (incluyendo semántico). También se puede dar nombre al índice y a la serie con `.index.name` y `.name`. Siempre hay 2 índices: posicional y semántico (si se define).

```
countries = pd.Series(['Spain', 'Andorra', 'Gibraltar', 'Portugal', 'France'],
                       index=[1, 2, 3, 4, 5])
print(countries)
```

```
0      Spain
1      Andorra
2      Gibraltar
3      Portugal
4      France
dtype: object
```

```
# especificando el índice semántico
countries = pd.Series(
    ['Spain', 'Andorra', 'Gibraltar', 'Portugal', 'France'],
    index=range(10, 60, 10)
)

print(countries)
```

```
10      Spain
20      Andorra
30      Gibraltar
40      Portugal
50      France
dtype: object
```

20.3. Slicing en Series

Cuando se utiliza un índice semántico, el slicing incluye ambos extremos del rango.

`reset_index(drop=..., inplace=...)` se utiliza para restablecer el índice de una `Series` o `DataFrame`.

- `drop=False`: Conserva el índice antiguo como una nueva columna del objeto.
- `drop=True`: Elimina el índice antiguo y crea un índice numérico estándar.
- `inplace=True`: Modifica el objeto original directamente y no devuelve uno nuevo.
- `inplace=False`: No modifica el objeto original y devuelve un nuevo objeto con el índice restablecido (comportamiento por defecto).

20.4. Iteración

```
for value in Fibonacci: # itera sobre valores
    pass

for index in Fibonacci.index: # itera sobre índices
    pass

for index, value in Fibonacci.items(): # itera sobre ambas
    pass
```

20.5. DataFrame

Datos tabulares: filas x columnas. Es como una serie pero con índices compartidos.

```
import pandas as pd
frame = pd.DataFrame(serie)
# display(frame) se ve mejor que print
```

Cambiar índice/columnas:

```
frame = pd.DataFrame(serie, index=["a", "b", "c"])
frame = pd.DataFrame(serie, columns=["Nombre", "edad", "sexo"])
```

Accesos y modificaciones:

```
frame["Nombre"] # para darle a la columna "Nombre" una valor
frame["Nombre"][0] # Para darle un valor a la primera fila de "Nombre"
frame["Direccion"] = lista_direcciones

# Añadir una fila (ejemplo conceptual):
frame.loc[3] = ["Juan", "32", "Hombre", "C\ ..."]

# Eliminar:
frame.drop("Nombre", axis=1, inplace=True)
frame = frame.drop(2) # fila
del frame["Nombre"]

frame.dtypes
```

Nulos y fechas:

```
df_con_nan = df.mask(df < 5, np.nan)
suma = matriz.isna().sum(axis=1) # Dice el número de Nan por fila
df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce") #
    Normaliza la fecha
```

Ejemplo de conversión a fecha con pd.to_datetime:

```
import pandas as pd

df = pd.DataFrame({
    "release_date": ["2023-01-15", "invalid_date", "2022-06-30"]
})

# Convierte la columna a tipo fecha
# Las fechas no válidas se convierten en NaT
df["release_date"] = pd.to_datetime(df["release_date"], errors="coerce")

print(df)
```

```
release_date
0    2023-01-15
1           NaT
2    2022-06-30
```

Iteración:

```
for a in frame:
    pass # columnas

for a in frame.values:
    pass # filas
```

Indexado:

```
print(frame.iloc[0]) # fila por índice posicional
print(frame.loc["a"]) # fila por índice semántico

print(frame.iloc[:, 0]) # columna por índice posicional
print(frame.loc[:, ["a"]]) # columna por índice semántico (ejemplo)
```

Cambiar índice:

```
frame.set_index("Edad", inplace=True)
```

20.6. Archivos en Pandas

- read_csv, to_csv, to_excel, to_parquet, read_sql
- Argumentos típicos: sep (obligatorio), index_col=0, engine="python"

20.7. Selección y filtrado, groupby

```
df[df["Edad"] > 30]

df.loc[0, "Nombre"]

df.groupby("Nombre")["Edad"].mean() # Agrupa por Nombre y calcula la media.
    Por ejemplo, calcula la media de todos los Pablo que estén en el array
    Nombre
```

Operaciones tras groupby:

- mean(), describe(), sum(), count(), size()
- min(), max(), median(), std()
- argmin(), argmax(), idxmin(), idxmax()
- agg({...}) múltiples operaciones

Ejemplo con .agg():

```
df.groupby("Categoria").agg(
    media_valor=("Valor", "mean"),
    suma_valor=("Valor", "sum"),
    cuenta=("Valor", "count")
)
```

Operaciones complejas:

```
"A" in frame # buscar nombres de columnas
frame.isin(3)
frame.apply(funcion, axis=1) # no se hacen bucles a dataframes
```

Agregaciones:

```
display(frame.groupby("genero")[["edad", "puntos"]].aggregate(["min", "mean", "max"]))
```

Filter:

```
# El metodo filter te permite hacer operaciones sobre los grupos y
    descartar aquellos que no cumplen una condicion determinada
display(count_por_genero)

frame2 = datos_por_genero.filter(
    lambda x: x["edad"].mean() > 30
)

display(frame2)
```

20.8. Correlaciones

```
frame["Speed"].corr(frame["generation"])
dataset.corr(numeric_only=True)
```

21. Visualization (Matplotlib)

```
data = np.random.randint(100, size=(3,))
display(data)

import matplotlib.pyplot as plt
plt.plot(lista # Crea el gráfico
plt.show() # Muestra el gráfico
```

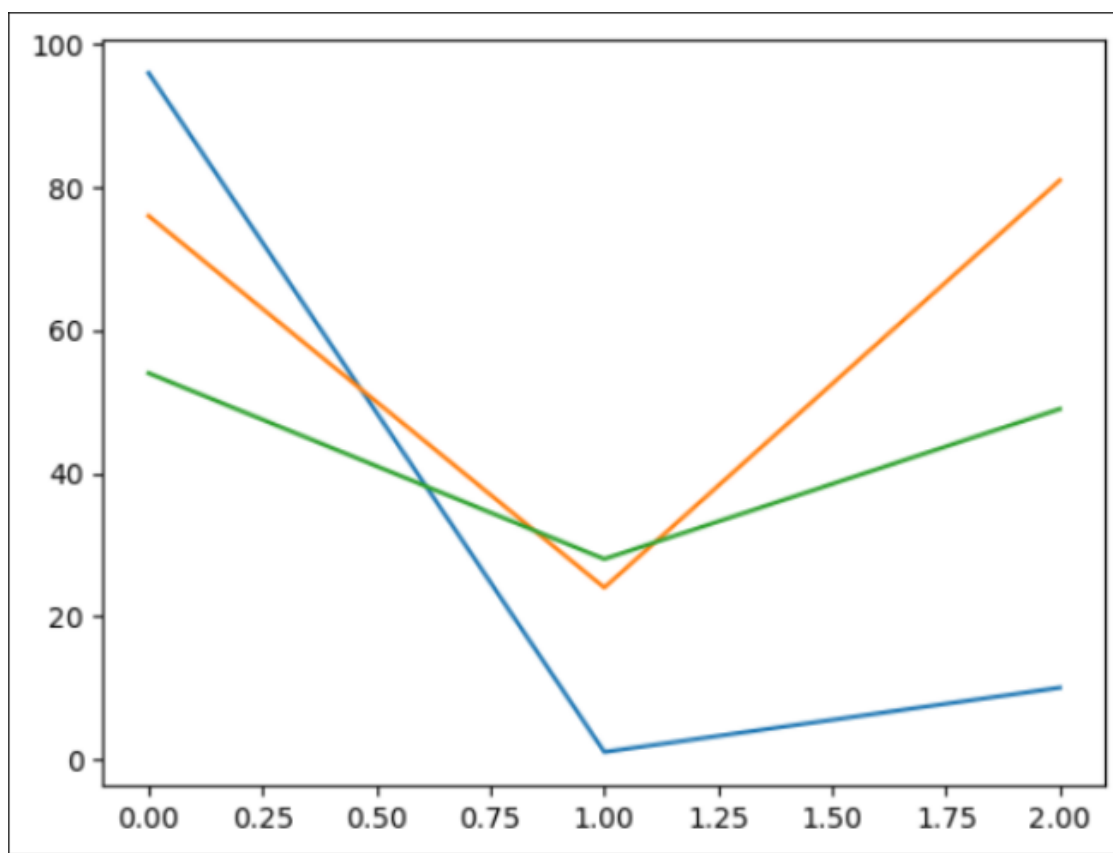


Figura 1: Ejemplo de gráfico

Ejemplo con ejes:

```
import numpy as np
x_data = np.arange(75, 100)
y_data = np.random.randint(80, size=(25))

if x_data.size == y_data.size:
    plt.plot(x_data, y_data)
    plt.show()
```

Figure y axes:

```
fig, axes = plt.subplots(1, 2) # 1 fila, 2 gráficos por fila
axes[0].plot(x_data1, y_data1) # 1º gráfico
axes[1].plot(x_data2, y_data2) # 2º gráfico
plt.show()
```

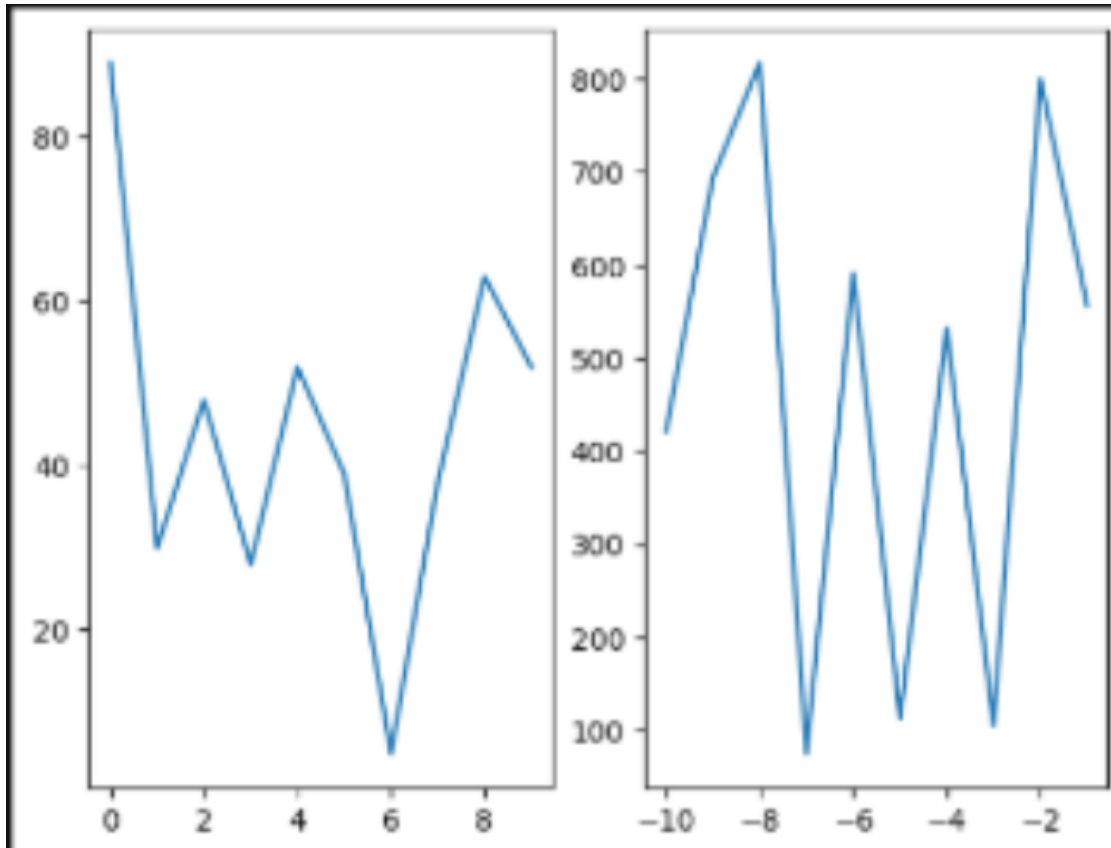


Figura 2: Ejemplo de gráfico

Configurar ejes:

```
axes[0].set_ylim([0, 1000])
axes[0].set_xlim([-1, 12])

axes[1].set_xlabel("Years")
axes[1].set_ylabel("Dollars", labelpad=-210, loc="top")
```

Compartir ejes:

```
# crear un nuevo eje con la x compartida
fig, ax = plt.subplots(1, 1)

x = np.arange(12)
ax.plot(x, "r")

ax2 = ax.twinx()
ax2.plot(x ** 2, "g")

plt.show()
```

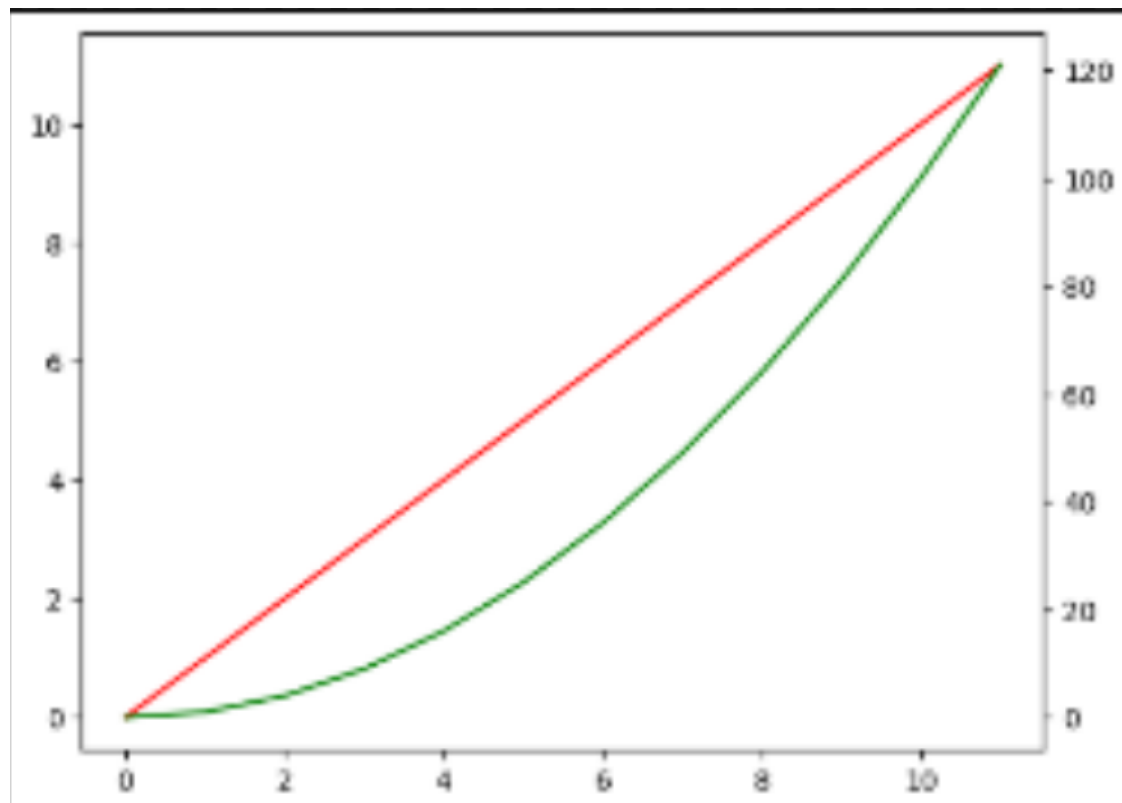


Figura 3: Ejemplo de gráfico

Parámetros de `plt.plot`: `plt.plot(x, y, style, label, linewidth=1, alpha=1)`

- `style`: [color] [marker] [line]
- `label`: leyenda
- `linewidth`: grosor de línea
- `alpha`: transparencia

Color	Descripción
b	blue
g	green
r	red
c	cyan
m	magenta
y	yellow
k	black
w	white

Cuadro 1: Códigos de color en Matplotlib

Marcador	Descripción
.	point marker
,	pixel marker
o	circle marker
v	triangle_down marker
^	triangle_up marker
<	triangle_left marker
>	triangle_right marker
1	tri_down marker
2	tri_up marker
3	tri_left marker
4	tri_right marker
s	square marker
p	pentagon marker
*	star marker
h	hexagon1 marker
H	hexagon2 marker
+	plus marker
x	x marker
D	diamond marker
d	thin diamond marker
	vline marker

Cuadro 2: Marcadores disponibles en Matplotlib

Línea	Descripción
-	solid line style
--	dashed line style
-.	dash-dot line style
:	dotted line style

Cuadro 3: Estilos de línea en Matplotlib

21.1. Representación de más de 1 gráfico en una subfigura

```
x_data = np.arange(10)

fig, axis = plt.subplots(1, 1)

axis.plot(x_data, np.random.randint(100, size=(10)), 'go-', label='green',
          linewidth=1, alpha=0.2) # creacion del grafico

axis.plot(x_data, np.random.randint(100, size=(10)), 'bx--', label='blue',
          linewidth=1) # creacion del grafico

axis.plot(x_data, np.random.randint(50, 80, size=(10)), 'rh-.', label='red',
          , linewidth=2, alpha=0.4) # creacion del grafico

axis.legend(loc='best')
plt.grid()

axis.set_xticks([0, 2, 3, 5, 7, 8], labels=['a', 'b', 'c', 'd', 'e'])
axis.set_title('Titulo')
```

```
axis.text(6, 90, 'Anotacion')
axis.set_frame_on(True)

plt.show()
```

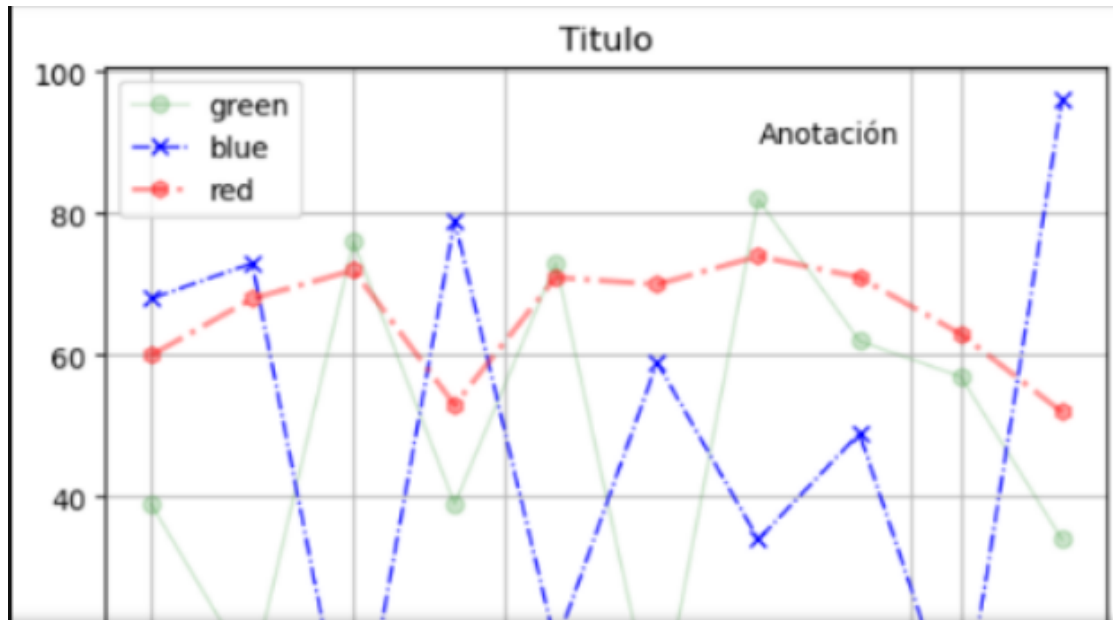


Figura 4: Ejemplo de gráfico

21.2. Anotaciones y tipos de gráficos

```
fig, axis = plt.subplots(1, 1)

axis.plot(x_data, y_data)

# axis.text(x_data[2], y_data[2], 'Low income')
# coordenadas igual a los datos

axis.annotate(
    'Low Income',
    xy=(x_data[2], y_data[2]),
    xytext=(4.70, 6),
    arrowprops=dict(facecolor='red', shrink=0.1)
) # anotacion completa

plt.show()
```

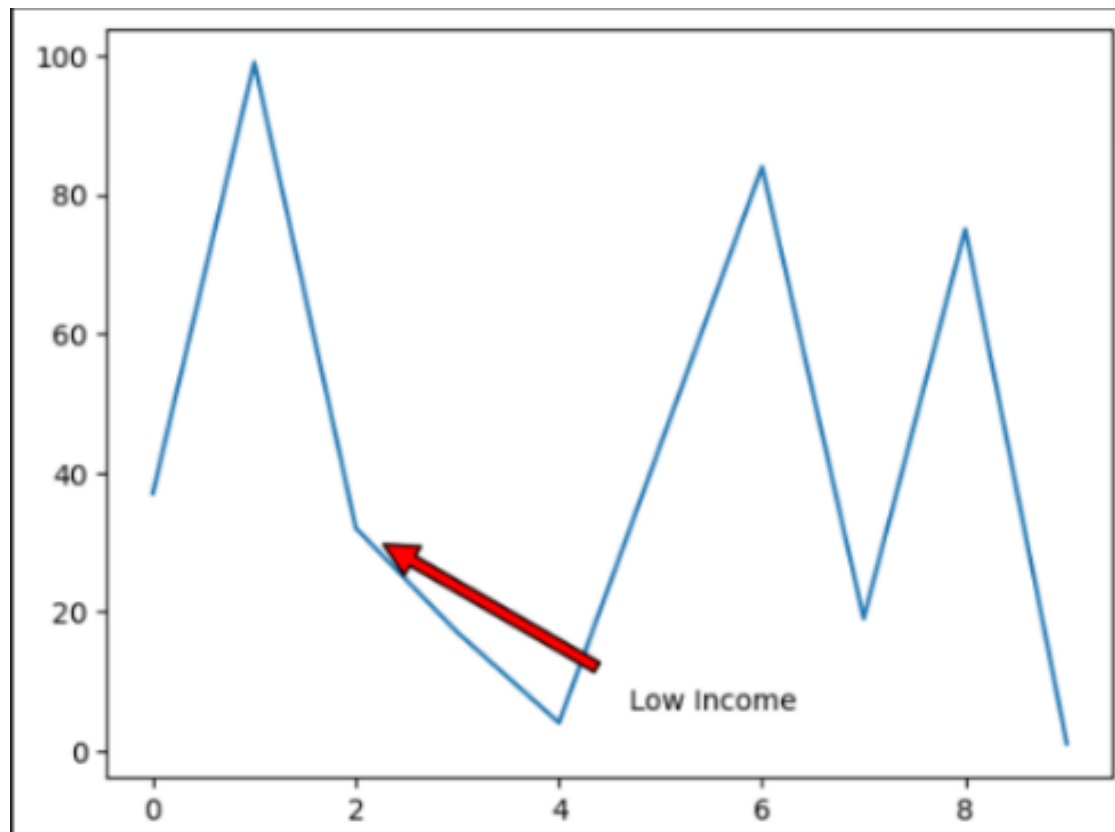


Figura 5: Ejemplo de gráfico

21.3. Tipos de gráficos

Barras

```
# Horizontal bar
fig, ax = plt.subplots()

# Example data
people = ('PhD', 'MSc', 'BSc', 'N/A')
y_pos = np.arange(len(people))
percentage = [5, 45, 25, 25]

ax.barh(y_pos, percentage, align='center', color='green', edgecolor='red')
#bar() para gráfico de barras verticales

ax.set_yticks(y_pos)
ax.set_yticklabels(people)
ax.invert_yaxis() # labels read top-to-bottom
ax.set_xlabel('Percentage of workforce')
ax.set_title('level of education in the company')

plt.show()
```

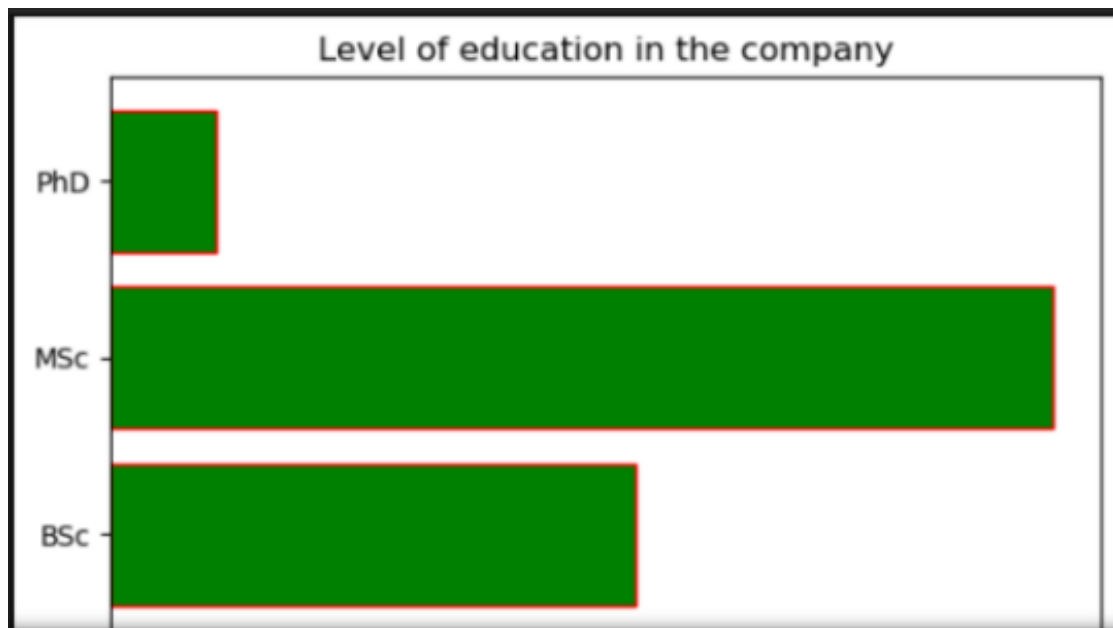


Figura 6: Ejemplo de gráfico

En barras, se pueden cambiar colores (ejemplo del PDF):

```
rects1[3].set_color('#872f29')
rects1[2].set_color('#ffcb59')
rects1[1].set_color('#d6ee39')
rects1[0].set_color('#6ade30')
```

Stack Plot

```
# stack plot
x = [2008, 2009, 2010, 2011, 2012]

iOS = np.array([35, 25, 20, 17, 18])
Android = [0, 15, 35, 45, 60]
WindowsPhone = [1, 3, 5, 5, 2]
Others = [64, 57, 40, 33, 20]

labels = ["iOS", "Android", "WindowsPhone", "Others"]

fig, ax = plt.subplots()

# Alternativa 1: Pasando en orden los datos del stack plot
# ax.stackplot(x, iOS, Android, WindowsPhone, Others, labels=labels)

# Alternativa 2: Pasando todos los datos en un np.array
y = np.vstack([iOS, Android, WindowsPhone, Others])
print(y)

ax.stackplot(x, y, labels=labels)

ax.set_xticks(x)
ax.legend(loc='upper left')

plt.show()
```

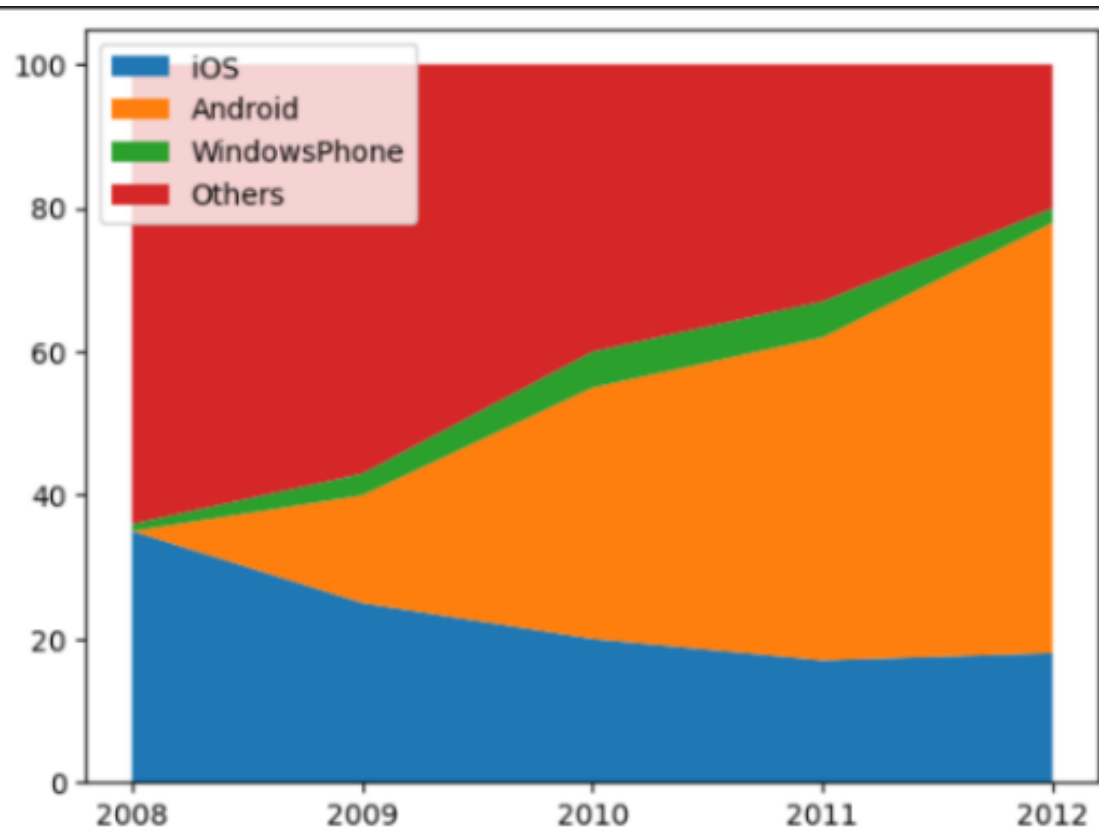


Figura 7: Ejemplo de gráfico

Scatter Plot

```
# Scatter plot
from numpy.random import rand

fig, ax = plt.subplots()
n = 700

for color in ['red', 'green', 'blue']:
    x, y = rand(2, n)
    scale = 25.0 * rand(n)
    ax.scatter(x, y, c=color, s=scale, label=color, alpha=0.4, edgecolors='none')

ax.legend()
plt.show()
```

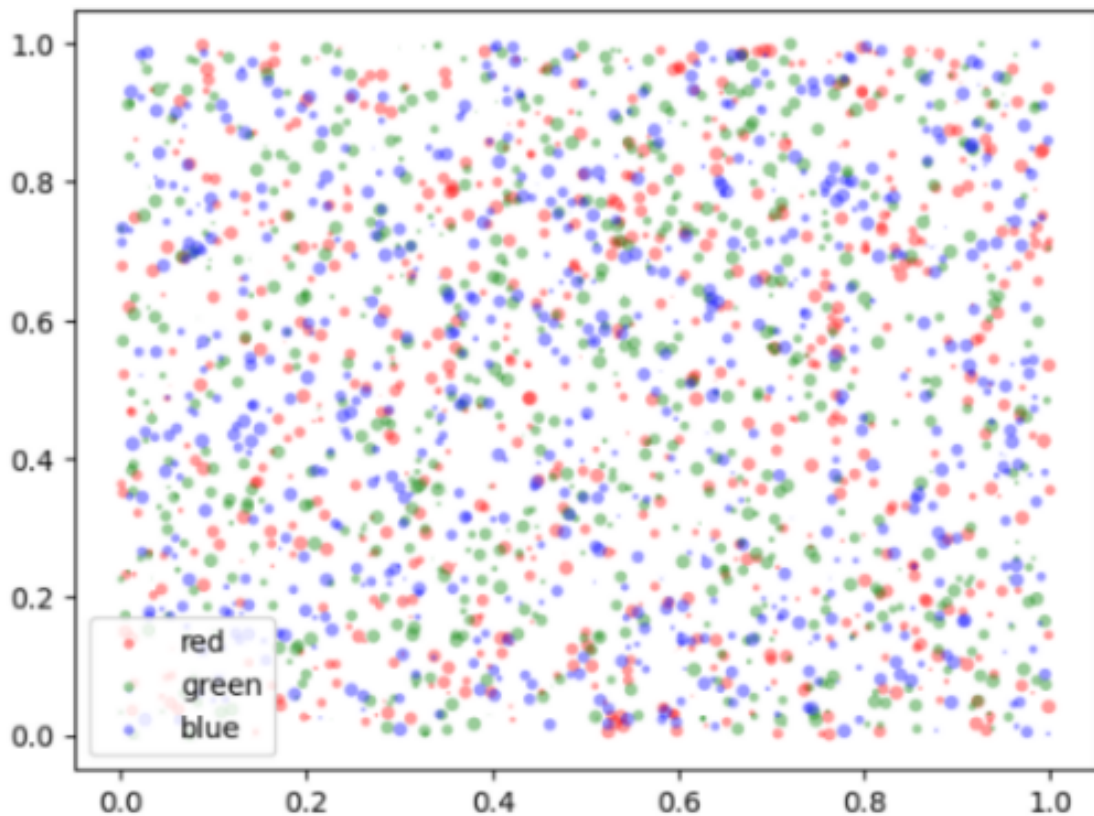


Figura 8: Ejemplo de gráfico

21.4. Pandas plotting

```
frame.plot()
frame.plot(x=..., y=..., label=..., ax=..., style=..., kind=..., xticks=
..., yticks=..., title=..., subplots=...)
plt.show()
```

Ejemplos:

```
frame.plot(kind='bar', title='Random lines')
plt.show()

frame.plot(kind='area', xticks=range(-2, 12), title='Random lines')

frame.hist()
```

Gráfico pastel:

```
import pandas as pd

df = pd.DataFrame(
    {'mass': [0.330, 4.87, 5.97],
     'radius': [2439.7, 6051.8, 6378.1]},
    index=['Mercury', 'Venus', 'Earth']
)

df.plot(y='radius', kind='pie', title='Planets')
```

22. Seaborn

La más sencilla

```
import seaborn as sns
import matplotlib.pyplot as plt

# Carga datasets de seaborn
# tips = sns.load_dataset("tips")

# Load data
flights_data = sns.load_dataset("flights")
# display(flights_data)

# Construct plot
sns.swarmplot(x="month", y="passengers", data=flights_data)

# Show plot
plt.xticks(rotation=45)
plt.show()
```

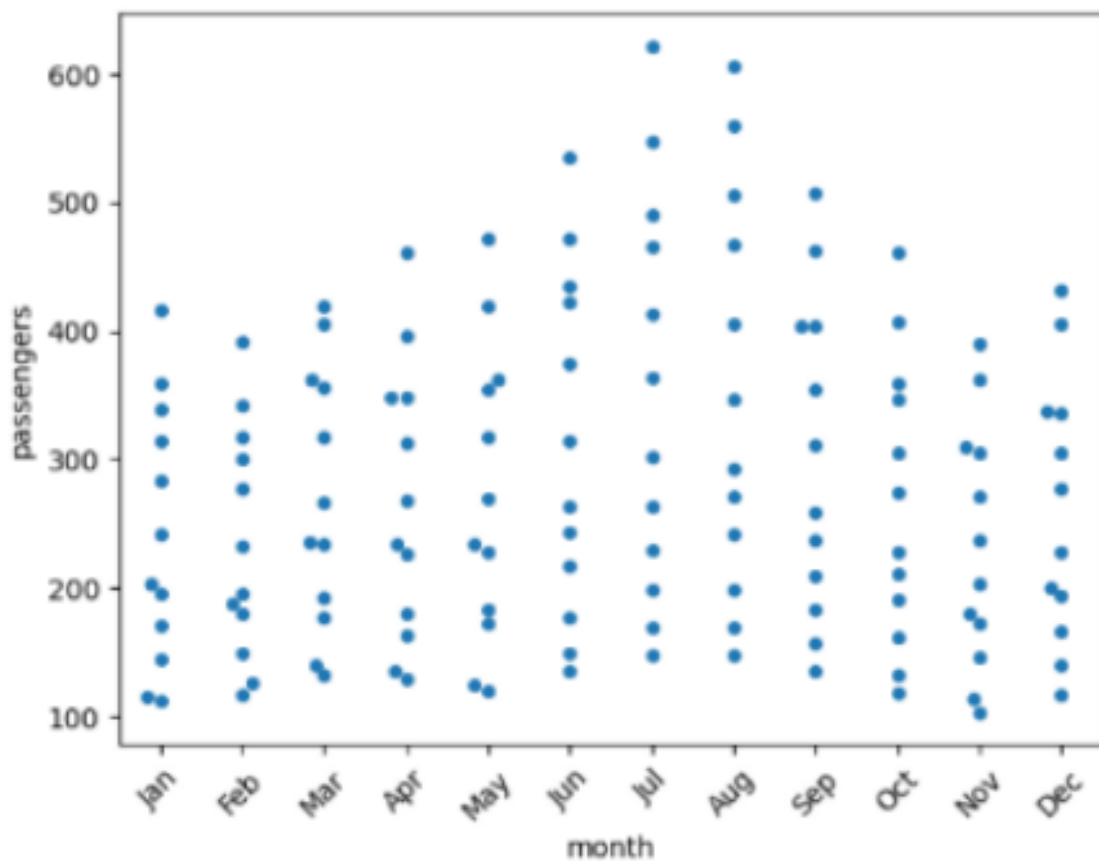


Figura 9: Ejemplo de gráfico

```
# Gráfico de barras
sns.barplot(x='sex', y='tip', hue='time', data=tips)
plt.show()
```

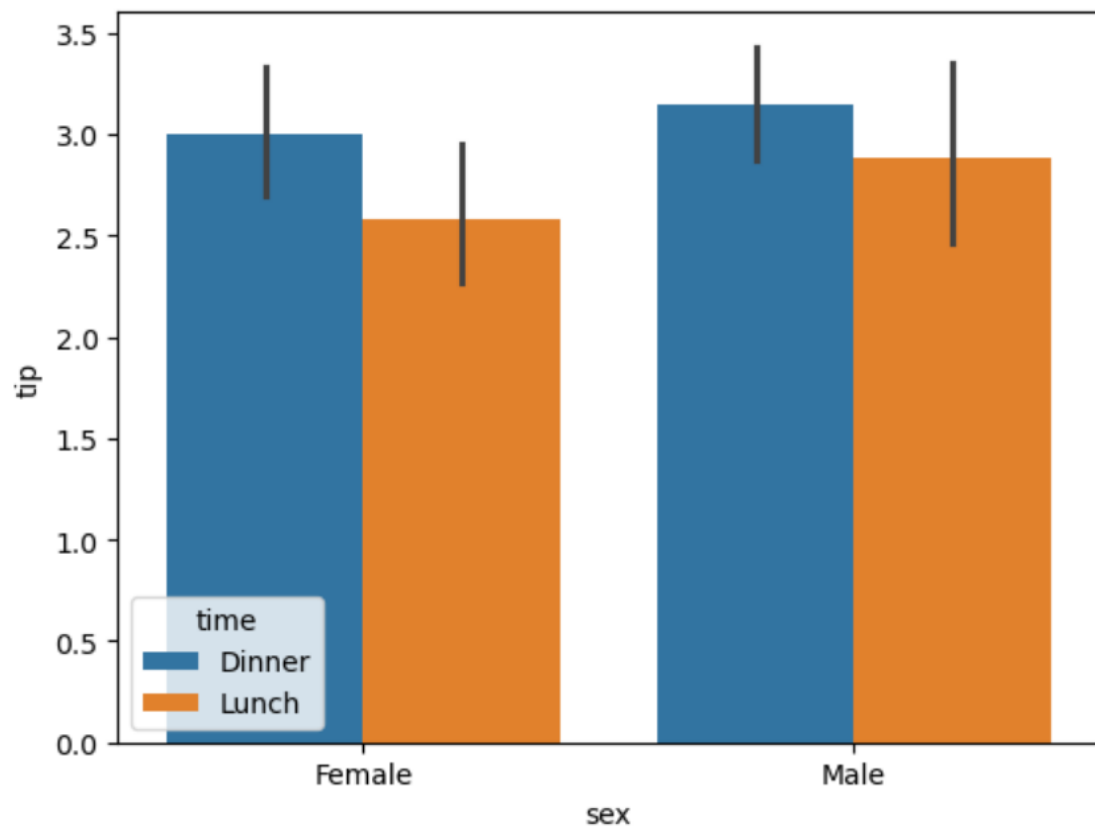


Figura 10: Ejemplo de gráfico

```
# Gráfico de cuartiles  
sns.boxplot(x=tips['day'], y=tips.tip)  
plt.show()
```

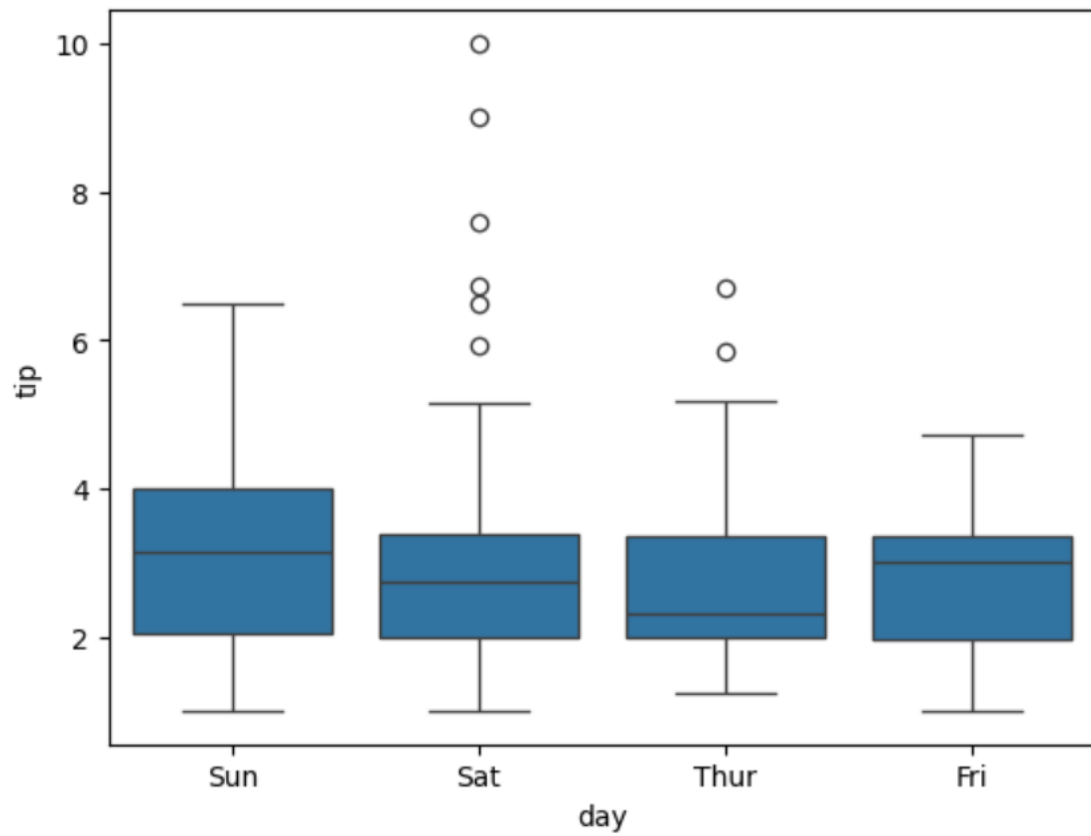



Figura 11: Ejemplo de gráfico

```
# Relaciones entre variables con scatter
sns.relplot(x='total_bill', y='tip', hue='sex', data=tips, kind='scatter')
plt.show()
```

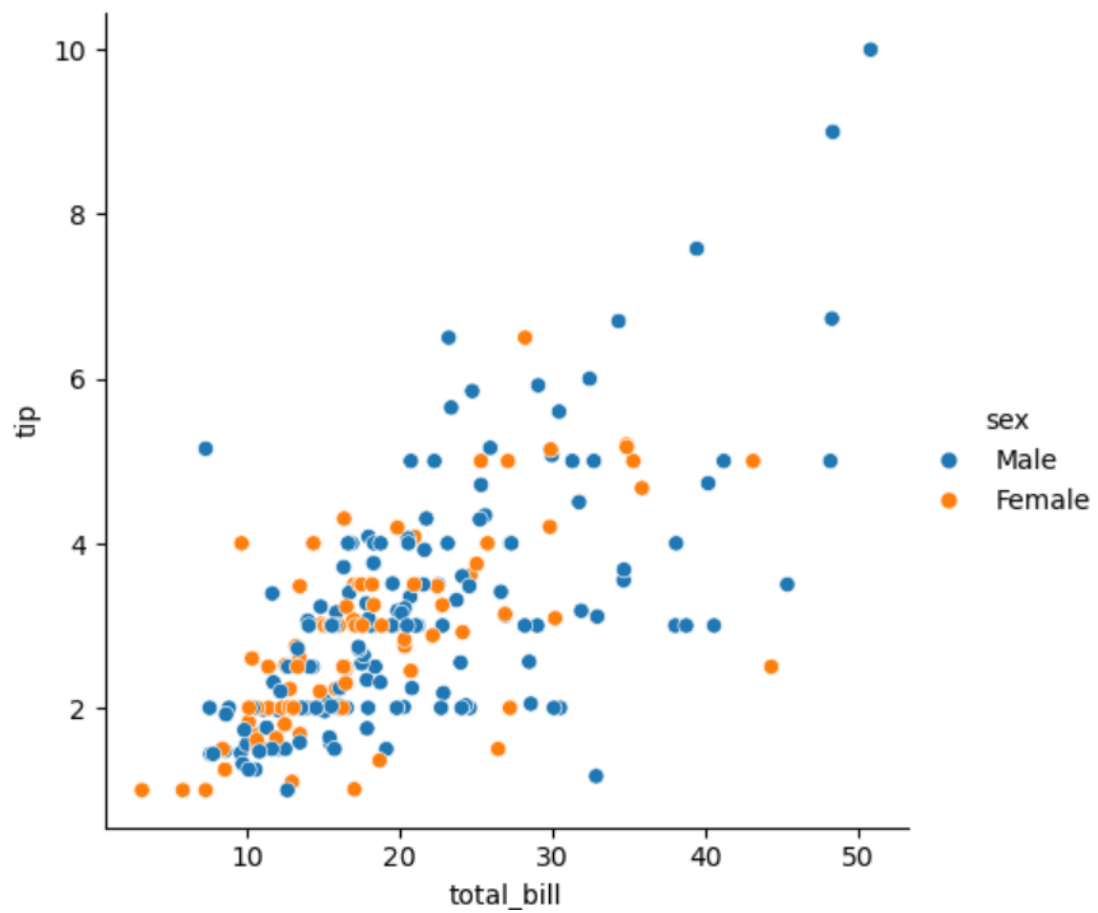


Figura 12: Ejemplo de gráfico

```
# Relaciones entre variables con line  
sns.relplot(x="year", y="passengers", kind="line", data=data)
```

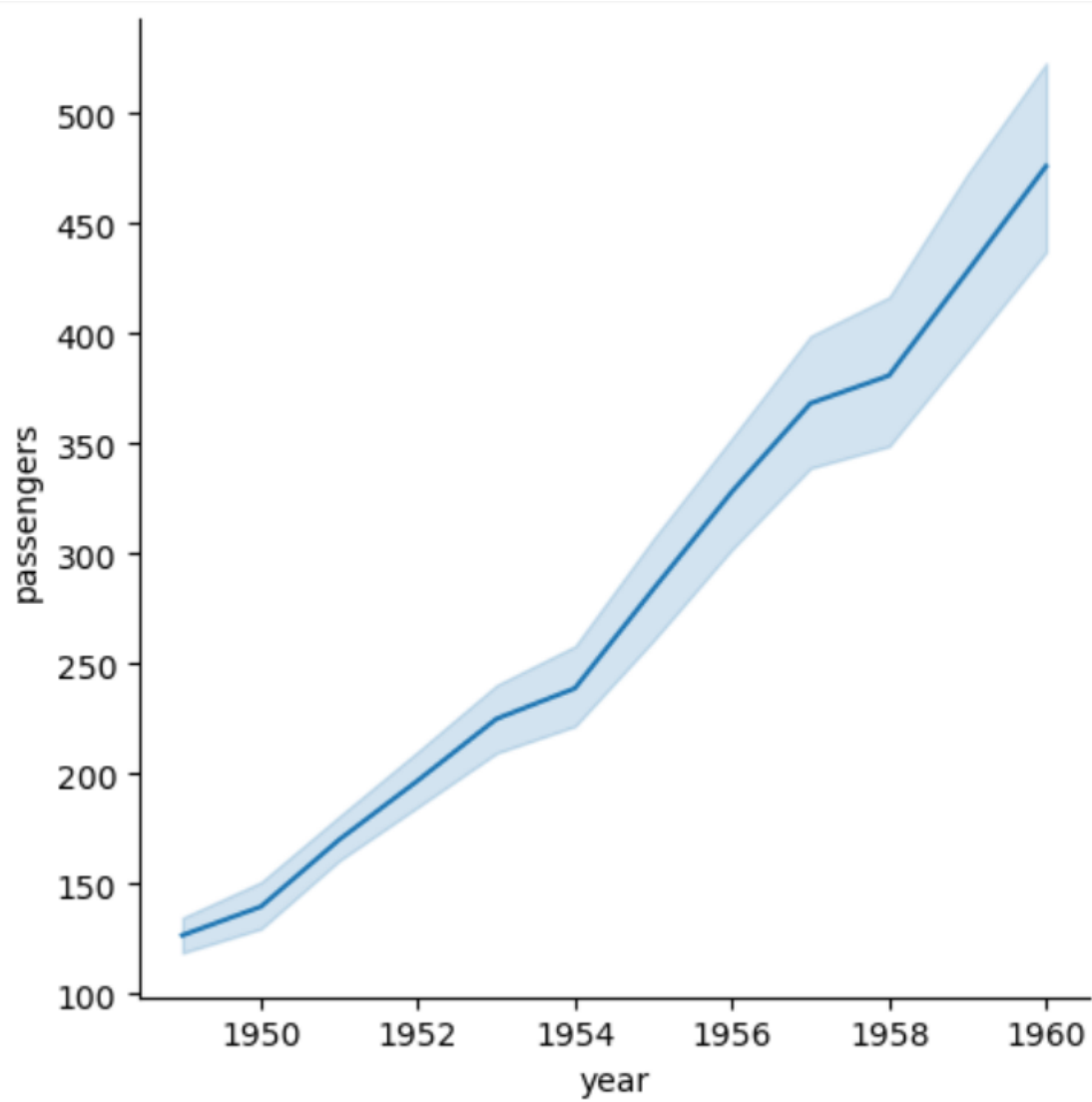


Figura 13: Ejemplo de gráfico

Distribución del dataset:

```
sns.displot(tips['tip'], kde=True, bins=10, rug=True)
plt.show()
```

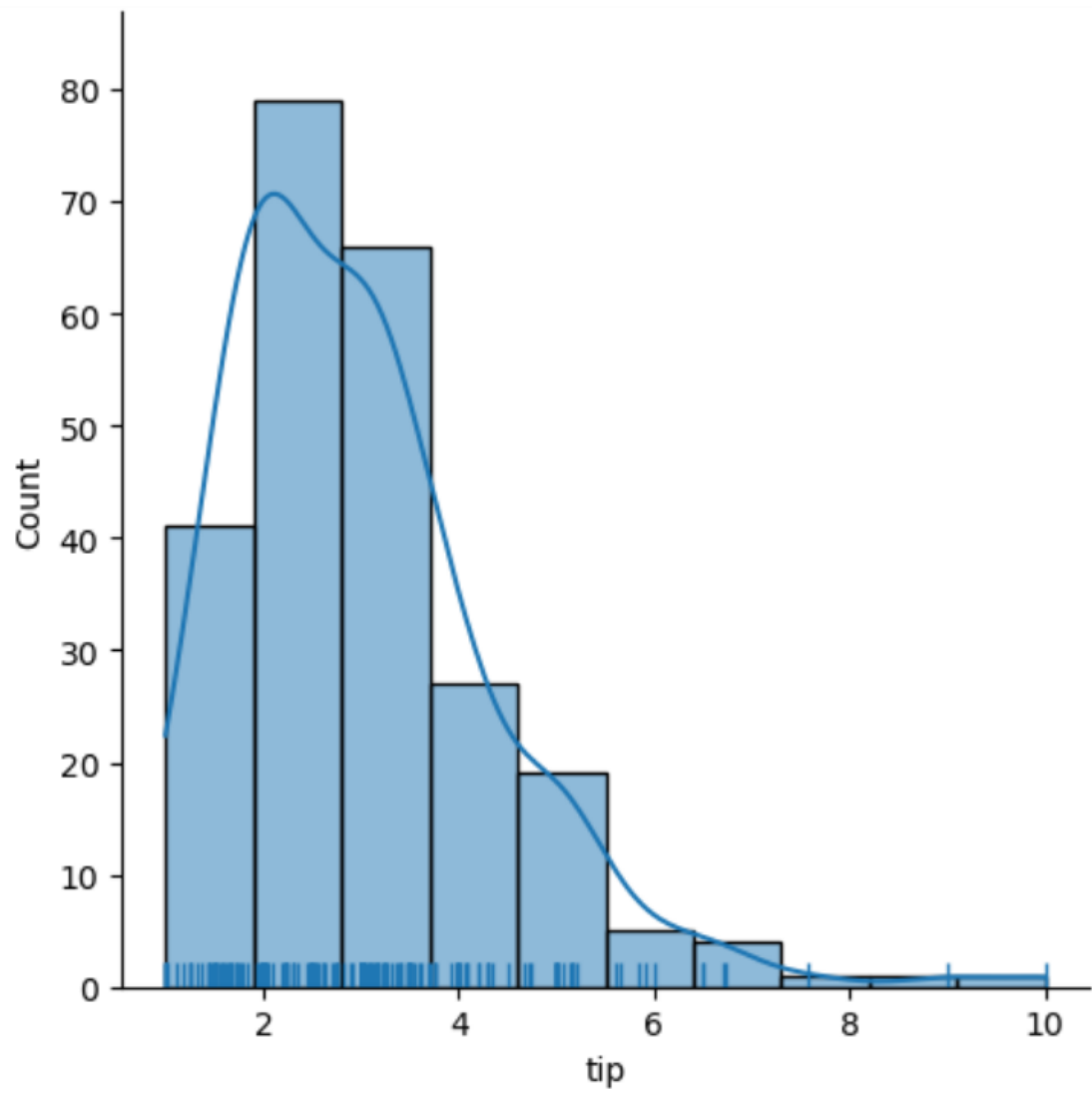


Figura 14: Ejemplo de gráfico

Distribución bi-variable:

```
sns.jointplot(x='total_bill', y='tip', data=tips)
plt.show()
```

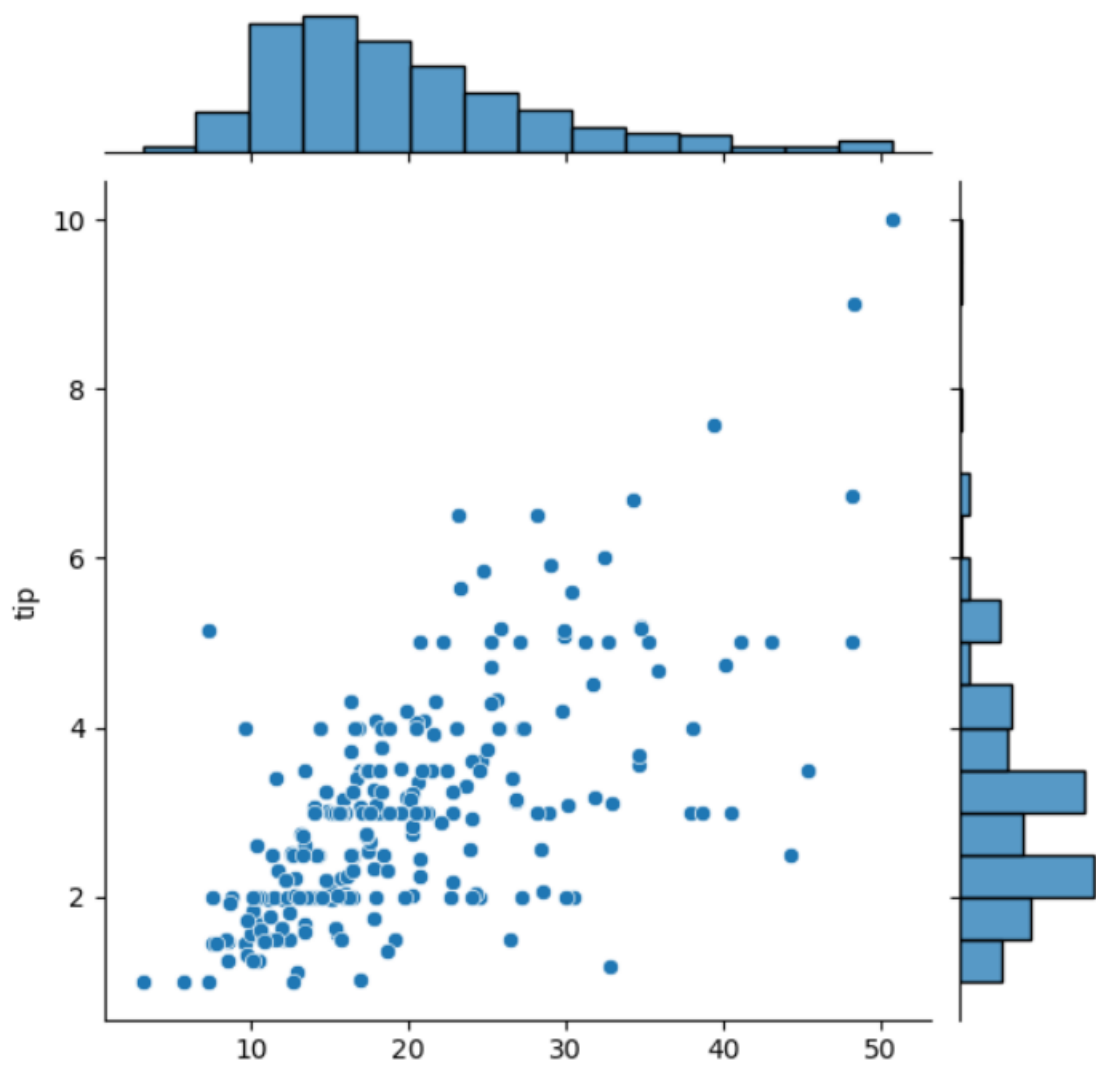


Figura 15: Ejemplo de gráfico

Densidad bivariantes:

```
iris = sns.load_dataset("iris")
sns.jointplot(x='petal_length', y='petal_width', kind='kde', data=iris)
plt.show()
```

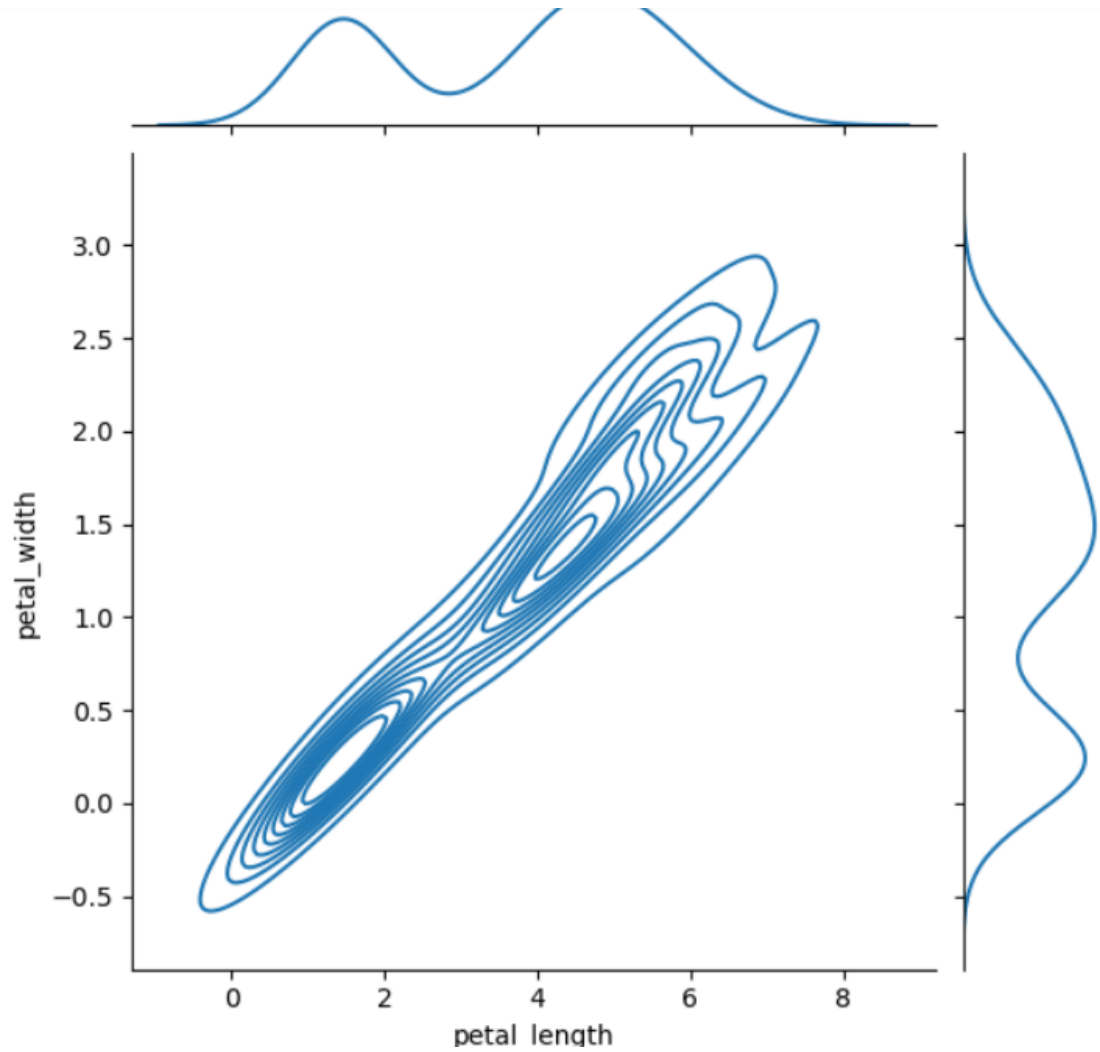


Figura 16: Ejemplo de gráfico

Pairplot:

```
sns.pairplot(iris)
sns.pairplot(iris, hue="species")
```

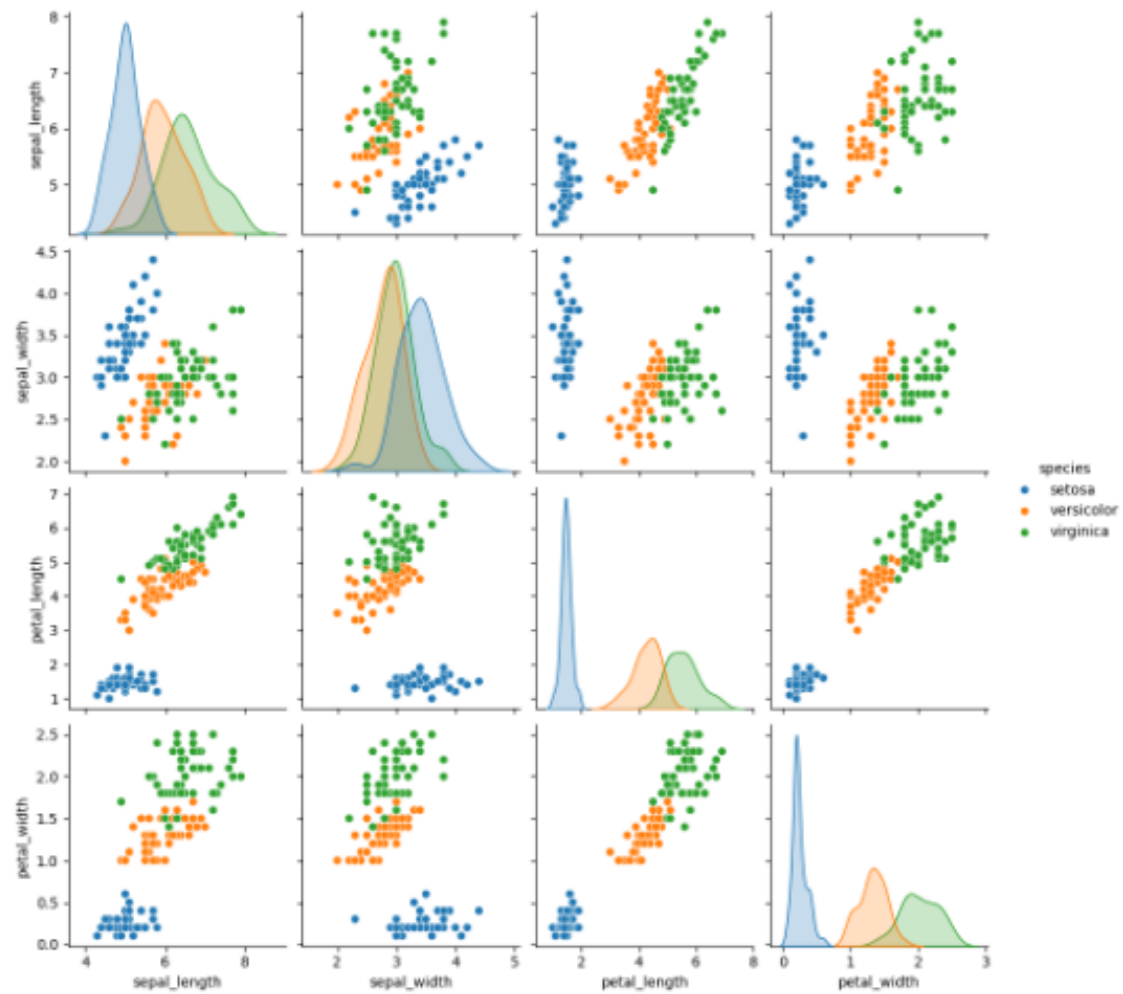


Figura 17: Ejemplo de gráfico

Correlaciones y regresión:

```
sns.regplot(x="total_bill", y="tip", data=tips) # Las variables deben ser
continuas, no discretas

sns.regplot(x="petal_length", y="sepal_width", data=iris, order=5)

sns.lmplot(x="total_bill", y="tip", hue="smoker", data=tips, col='time',
order=1)
```

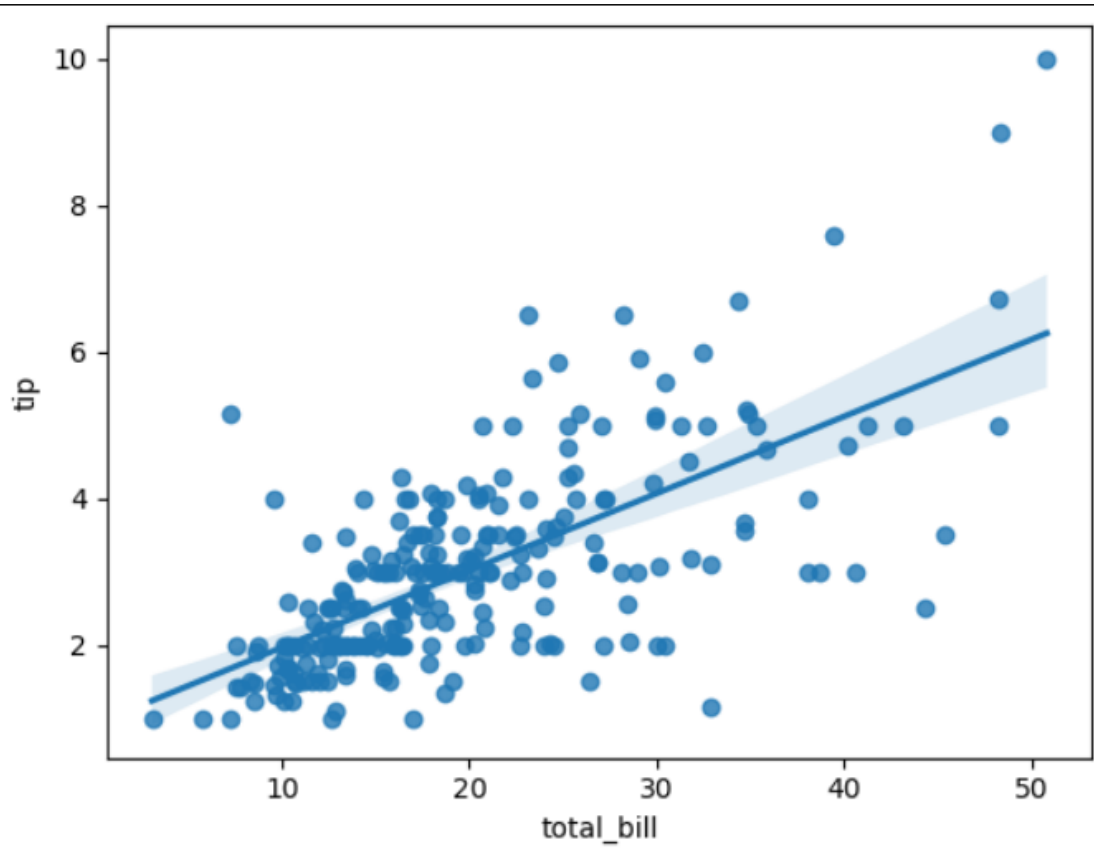


Figura 18: Ejemplo de gráfico continuo de 1 grado

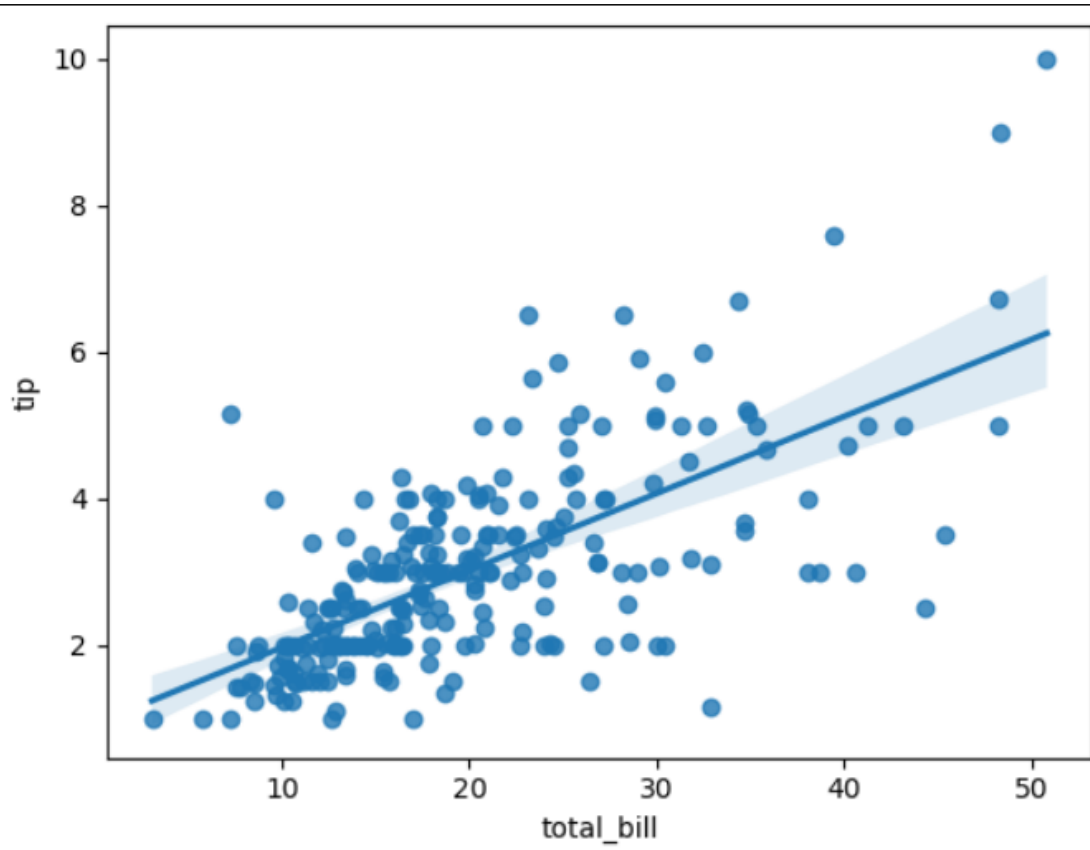


Figura 19: Ejemplo de gráfico continuo de 5 grado

23. Procesamiento de datos

23.1. Combinar datasets

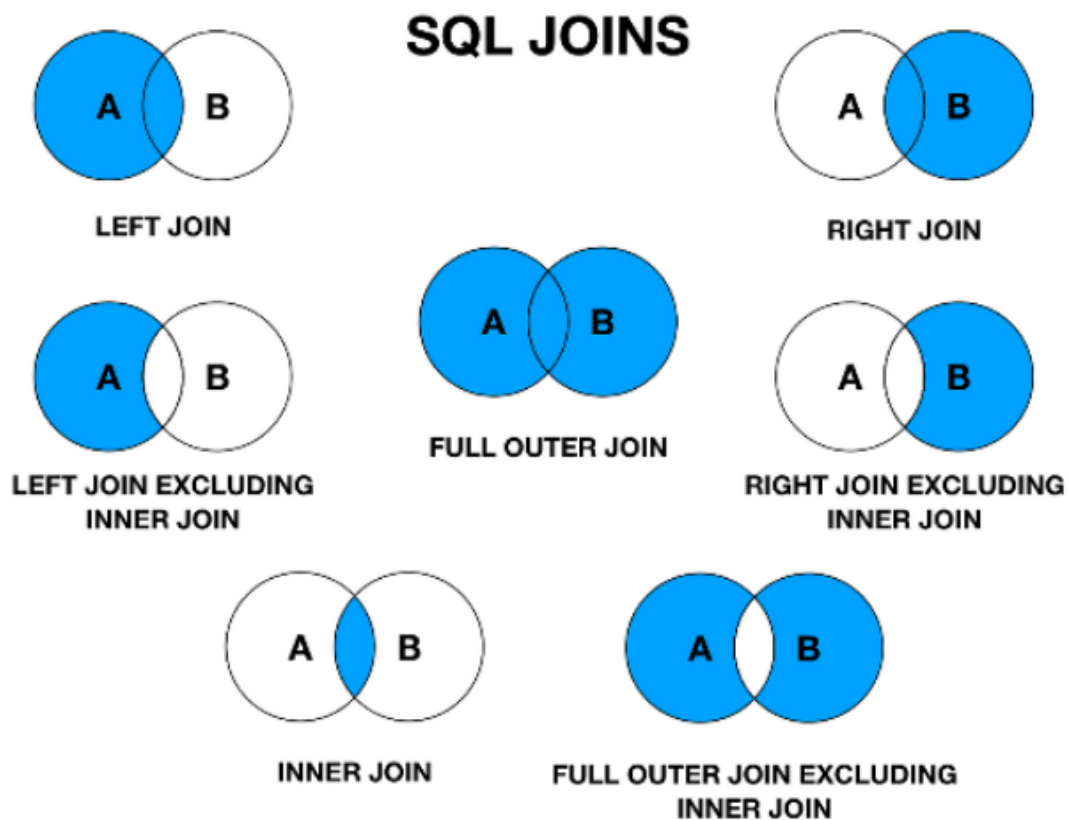


Figura 20: Ejemplo de gráfico

`join() == merge()`. Según el PDF: `join` usa el índice como pivote; en `merge` se especifica (recomendado usar `merge`).

Ejemplo:

```
combined_dataset = users_dataset.merge(ratings_dataset, on="UserID", how="inner")
```

- `on="UserID"` columna pivote
- `how="inner"` si un `UserID` no está en B, no se une

Representar con `pivot_table`:

```
display(all_dataset.pivot_table('Rating', index='Gender', columns='Age'))

display(all_dataset.pivot_table(
    'Rating',
    index='Gender',
    columns='Age',
    aggfunc=['count', 'max', 'mean']
))
```

23.2. Agrupaciones

`.unique()` saca valores únicos.

23.3. Manipulación de strings

`split("a")` separa texto por carácter. Nota del PDF: `AttributeError` si aplicas `split` a una lista de listas (no a strings).

23.4. Expresiones regulares

Para patrones en strings (según PDF: complicadas).

23.5. Operaciones con colecciones: `functools`

- `reduce`: aplica operación acumulativa y retorna un valor
- `filter`: retorna secuencia que cumple condición
- `map`: aplica operación y retorna secuencia

24. Transformación de variables (Pandas/DataFrame)

- `np.nan == NaN`
- `isnull()` para saber si es 0
- `isna()` para saber si es nulo
- `count()` valores no nulos
- `dropna(thresh=4, axis=1)` elimina las columnas con más de 4 Nan
- `fillna(-1)` cambia los nan por -1
- `bfill()` cambia los nan por el valor de la fila de abajo, `ffill()` cambia el nan por el valor de la fila de arriba
- `interpolate()` cambia el nan por interpolación entre la fila de arriba y abajo

24.1. Duplicados

```
serie.duplicated() # Devuelve todos los valores de la serie junto con un
                  true o false si se repiten o no.

df.drop_duplicates(subset="Gender", keep="first", inplace=False) # elimina
                        duplicados de una columna, quedándose solo con el primero
```

25. Examen: Generadores

Resumen (según PDF): producen valores uno a uno bajo demanda, sin almacenar todo en memoria, útiles para eficiencia, legibilidad y escalabilidad con grandes volúmenes o flujos.

(En el PDF aparece la comparación “Función normal vs función generadora”).

26. For en una línea

```
# 1. for simple con una instruccion
for i in range(5): print(i)

# 2. List comprehension (crear una lista)
cuadrados = [i**2 for i in range(5)] # [0, 1, 4, 9, 16]

# 3. Con condicion
pares = [i for i in range(10) if i % 2 == 0] # [0, 2, 4, 6, 8]

# 4. Diccionario por comprension
cuadrados = {i: i**2 for i in range(5)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# 5. Varias acciones (usando ;)
for i in range(3): print(i); print(i**2)
```